

HPE iMC -> NetBox - laboratório e produção por etapas

Estado de entrega: preparação offline

O projeto está preparado para início futuro. Não há APIs do iMC ou do NetBox disponíveis nesta fase e elas não são necessárias para validar o código localmente. As etapas operacionais abaixo serão executadas quando o ambiente estiver disponível.

Para conferir a entrega agora, sem configurar credenciais nem acessar serviços:

```
.venv/bin/python scripts/validate.py
```

O verificador usa respostas simuladas, não conecta usando iMC/.env e grava o resultado em artifacts/validation.json. Em uma máquina nova, instale Python e as dependências conforme a seção 1; a validação completa também usa Git, Bash e pdftotext do pacote poppler-utils.

Consulte o guia de início do projeto (docs/INICIO_DO_PROJETO.md) e o relatório de validação atual (docs/VALIDACAO_ETAPAS.md). A entrega já está incorporada à pasta principal e não depende da cópia temporária da sessão anterior.

A integração lê o inventário do iMC, compara com o NetBox e altera somente descrição e VLANs das interfaces que já existem no NetBox. O laboratório usa um switch escolhido pelo operador. Em produção, todos os dispositivos do iMC são consultados e comparados.

Um dispositivo que existe no iMC e não existe no NetBox é descartado, registrado pelo nome nos logs e contado em devices_not_found. Isso é esperado e não gera erro de execução. As interfaces desse dispositivo não são consultadas. Nenhum dispositivo é criado ou alterado.

Este é o README oficial de operação. Execute cada etapa separadamente e confira o resultado antes de avançar. Não há requisito de cadastrar uma lista fixa de switches para produção.

1. Preparar Python e dependências

No Linux Ubuntu/Debian:

```
python3 --version
# Se necessário:
sudo apt update
sudo apt install python3 python3-venv python3-pip util-linux
```

Entre na raiz desta cópia do projeto (ajuste o caminho se necessário):

```
cd /home/willia/Downloads/Sincronizacao-Netbox-IMC-HPE-main
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

```
python -m pip check
```

Python 3.10 ou superior. requests é usado pelas APIs; pytest pelos testes offline. Não reutilize um ambiente virtual copiado de outra máquina. Para uma sessão nova, basta entrar na raiz e executar `source .venv/bin/activate`. Para sair: `deactivate`.

2. Configurar o ambiente local

```
# Só copia se .env ainda não existir; preserva credenciais já preenchidas.
test -f imc/.env || cp imc/.env.example imc/.env
chmod 600 imc/.env
nano imc/.env
```

Cada script carrega `imc/.env` automaticamente como dados. Não execute `source imc/.env`. Valores com espaços precisam de aspas. Variáveis exportadas no terminal prevalecem sobre o arquivo; `IMC_ENV_FILE` permite escolher outro arquivo local.

Variável	Uso
IMC_HOST	Hostname ou IPv4, sem protocolo ou porta
IMC_PORT	Porta real do iMC; padrão 8443 com HTTPS
IMC_USE_HTTPS	true para HTTPS; false para HTTP explicitamente configurado
IMC_USERNAME / IMC_PASSWORD	Conta de serviço para leitura
IMC_VERIFY_SSL	true; em HTTPS utilize a CA correta
IMC_CA_BUNDLE	Opcional: caminho da CA interna
NETBOX_URL	Origem do NetBox, sem /api
NETBOX_TOKEN	Token de leitura; alteração de interfaces somente na aplicação
NETBOX_VERIFY_SSL / NETBOX_CA_BUNDLE	Validação TLS e CA interna
NETBOX_VLAN_GROUP_ID	Opcional: ID de um grupo de VLAN explicitamente escolhido
REQUEST_TIMEOUT	Timeout por requisição, de 1 a 300 segundos; padrão 30

As primeiras três etapas precisam somente das variáveis do iMC. O teste do NetBox precisa somente das variáveis do NetBox. Comparação, simulação e aplicação precisam dos dois serviços. Não é necessário ter configurado o NetBox para começar a descoberta do iMC.

3. Scripts para cada etapa

Todos os comandos abaixo são executados na raiz do projeto, com o ambiente virtual ativo. SW-LAB-01 é um exemplo: substitua pelo nome exato obtido na listagem.

Etapas	Script	Efeito
1. Conexão iMC	<code>testar_conexao_imc.py</code> (<code>imc/testar_conexao_imc.py</code>)	Um GET autenticado, sem inventário completo
2. Descoberta	<code>listar_dispositivos_imc.py</code> (<code>imc/listar_dispositivos_imc.py</code>)	Lista IDs e nomes de todos os devices, somente GET
3. Inspeção de um switch	<code>inspecionar_switch_imc.py</code> (<code>imc/inspecionar_switch_imc.py</code>)	Lê interfaces, descrições e VLANs do switch escolhido

4. Conexão NetBox	`testar_conexao_netbox.py` (`imc/testar_conexao_netbox.py`)	Testa GET em dispositivos, interfaces e VLANs
5. Comparação de inventários	`comparar_dispositivos.py` (`imc/comparar_dispositivos.py`)	Compara nomes e registra encontrados/ausentes, sem ler interfaces
6. Simulação do laboratório	`simular_switch.py` (`imc/simular_switch.py`)	Planeja diferenças de um switch; zero PATCH
7. Aplicação do laboratório	`aplicar_switch.py` (`imc/aplicar_switch.py`)	Recalcula o plano, pede SIM e aplica diferenças do switch
8. Produção	`sincronizar_producao.py` (`imc/sincronizar_producao.py`)	Compara todos; dry-run padrão, aplicação explícita

Etapa 1 - conexão com o iMC

```
python imc/testar_conexao_imc.py
```

Etapa 2 - listar os dispositivos

```
python imc/listar_dispositivos_imc.py
```

Confira o nome do switch que será usado no laboratório. Essa etapa não seleciona nem aplica alterações automaticamente.

Etapa 3 - ler o switch selecionado

```
python imc/inspecionar_switch_imc.py --device "SW-LAB-01"
```

O inventário é lido para resolver o nome, mas somente as interfaces desse switch são consultadas. Nome ausente ou duplicado impede a inspeção. Confira descrição, `ifIndex`, modo, PVID e VLANs com a configuração real do switch. Resultado inconclusivo fica registrado.

Etapa 4 - testar acesso ao NetBox

```
python imc/testar_conexao_netbox.py
```

Confirma leitura dos três tipos de objeto. Um GET bem-sucedido não comprova permissão de PATCH.

Etapa 5 - comparar os inventários

```
python imc/comparar_dispositivos.py
```

Mostra correspondências, dispositivos ausentes e ambiguidades. O nome precisa coincidir exatamente. Não há fallback por IP, abreviação ou posição na lista. Ausência é descarte normal; ambiguidade ou erro de comunicação exige revisão.

Etapa 6 - simular somente um switch

```
python imc/simular_switch.py --device "SW-LAB-01"
```

Revise os campos anteriores e propostos. O script filtra o switch antes de consultar suas interfaces ou procurar os demais devices no NetBox. O resultado dessa etapa é uma proposta nos logs, não um arquivo executável de mudanças.

Etapa 7 - aplicação controlada

```
python imc/aplicar_switch.py --device "SW-LAB-01"
```

Use depois de validar o dry-run. A aplicação relê os dados, mostra um plano novo e solicita SIM em um terminal interativo. Confira novamente: o plano pode ter mudado desde a simulação. Não aceita aplicação em todos os switches nem confirmação não interativa por essa entrada de laboratório.

Etapa 8 - produção, após aprovação e validação

Simular todo o inventário:

```
python imc/sincronizar_producao.py --dry-run
```

Aplicar com confirmação:

```
python imc/sincronizar_producao.py --apply
```

Aplicar sem prompt, quando a operação não interativa já fizer parte do procedimento aprovado:

```
python imc/sincronizar_producao.py --apply --non-interactive
```

O script lê todos os dispositivos do iMC, procura nomes únicos no NetBox e processa somente os correspondentes. Devices ausentes são documentados e descartados. Ele não cria inventário nem modifica objetos de dispositivo: o PATCH atinge exclusivamente interfaces existentes.

4. Regras das VLANs e descrições

- Campos permitidos: description, mode, untagged_vlan e tagged_vlans.
- Nenhum POST, PUT ou DELETE. Sem alteração de enabled, status, IP, MAC ou tipo.
- Interface ausente, VLAN ausente ou correspondência ambígua bloqueia aquela interface e aparece no resumo.
- NETBOX_VLAN_GROUP_ID restringe a busca a um grupo explicitamente escolhido e valida o grupo no retorno. Sem ele, o VID precisa ser único em todo o NetBox. O script não escolhe um site/grupo automaticamente.
- Access exige correspondência pelo ifIndex, uma única VLAN e PVID igual a essa VLAN. Falha nos endpoints trunk/hybrid é registrada e não bloqueia access confirmado pelos seus próprios dados; conflitos encontrados nas respostas disponíveis continuam bloqueados.
- Trunk exige classificação completa e PVID presente na associação; o código

representa PVID como nativa e demais VLANs como tagged. Essa interpretação ainda precisa ser validada com respostas reais do iMC instalado.

- Hybrid e consultas vazias/inconclusivas continuam bloqueados. O código não limpa associações por falha de consulta.
- Uma lista tagged pode ser limpa quando uma configuração access válida confirma a transição.
- Descrição vazia ou null explicitamente recebida pode limpar a descrição atual. Campo ausente ou inválido preserva a descrição. Se a VLAN estiver inconclusiva, a interface inteira é preservada, inclusive a descrição.

5. Confirmação da escrita e logs

Antes de cada PATCH, o NetBox é relido para detectar mudanças desde o planejamento. Depois do PATCH, um novo GET confere ID e campos propostos. Somente o estado confirmado aumenta applied.

Timeout ou perda de conexão durante PATCH é resultado incerto. O script faz GET de reconciliação e não repete o PATCH automaticamente. Se os campos estiverem corretos, confirma o resultado; se não for possível confirmar, registra erro e unconfirmed. A checagem anterior reduz conflitos, mas GET/PATCH não são uma transação atômica.

Logs: logs/pilot.log, com UUID da execução, horário UTC, modo, escopo, grupo VLAN, nomes consultados, propostas e resultados. Rotação de 5 MB com cinco backups; diretório restrito e mascaramento de credenciais. Os logs contêm inventário e descrições: mantenha-os no ambiente autorizado.

```
tail -n 100 logs/pilot.log
```

```
# Para acompanhar:
```

```
tail -f logs/pilot.log
```

Contador	Significado	
---	---	
devices / devices_found	Dispositivos retornados pelo IMC / correspondentes no NetBox	
devices_filtered	Fora do switch selecionado, no laboratório	
devices_not_found	Ausentes no NetBox, descartados sem erro	
devices_skipped / interfaces_skipped	Descartes separados por tipo	
interfaces / interfaces_found / interfaces_not_found	Interfaces lidas, correspondentes e ausentes	
vlans_not_found	Interfaces bloqueadas por VLAN não encontrada	
ambiguities	Ocorrências de correspondência ambígua	
synchronized / divergent	Interfaces iguais / com diferenças válidas	
planned / interfaces_simulated	Propostas válidas / propostas em dry-run	
accepted / applied	PATCHes aceitos por HTTP / alterações confirmadas por GET	
uncertain / unconfirmed	Envios com resultado incerto / tentativas não confirmadas	
errors / duration_seconds	Erros de processamento / duração do fluxo	

skipped é mantido por compatibilidade como total de descartes. As categorias podem se

sobrepor: um timeout pode ser reconciliado e depois confirmado.

Código de saída: 0 sem erros (pode incluir dispositivos ausentes descartados ou cancelamento pelo operador); 1 falhas por objeto/resultado inconclusivo; 2 configuração, argumentos, lock ou falha global. Use `echo $?` imediatamente após o comando. Na inspeção, 1 indica VLANs inconclusivas.

6. Testes offline

```
python -m pytest -q
python scripts/mock_dry_run.py
python -m compileall -q imc scripts tests sincronizacao_imc_netbox.py
for script in imc/*.sh; do bash -n "$script"; done
git diff --check
```

A suíte bloqueia transporte de rede real. Cobre seleção do laboratório, leitura de todos em produção, ausência normal de devices, limites de método/campo, grupo VLAN, falhas de endpoints, confirmação posterior e timeout sem reenvio. Mocks não comprovam conectividade, permissões ou semântica das respostas reais.

7. Núcleo e scripts auxiliares

Os scripts de etapa compartilham `etapas.py` (`imc/etapas.py`), `sync.py` (`imc/sync.py`), os clientes iMC (`imc/imc_client.py`) e NetBox (`imc/netbox_client.py`), `policy.py` (`imc/policy.py`) e `runtime.py` (`imc/runtime.py`). Isso mantém as regras iguais em todas as etapas.

A entrada compatível `sincronizacao_imc_netbox.py` (`sincronizacao_imc_netbox.py`) permanece. `--apply` nela exige escopo explícito:

```
python sincronizacao_imc_netbox.py --device "SW-LAB-01" --dry-run
python sincronizacao_imc_netbox.py --device "SW-LAB-01" --apply
python sincronizacao_imc_netbox.py --all-devices --dry-run
```

`executar_sincronizacao.sh` (`imc/executar_sincronizacao.sh`) repassa argumentos e usa `flock`. Para usar o Python correto fora da sessão interativa:

```
IMC_PYTHON_BIN="$PWD/.venv/bin/python" bash imc/executar_sincronizacao.sh --device
"SW-LAB-01" --dry-run
```

`instalar_cron.sh` (`imc/instalar_cron.sh`) continua desativado até validar a operação. `desinstalar_cron.sh` (`imc/desinstalar_cron.sh`) remove o agendamento legado do usuário; não faz parte dos testes. Todos os fluxos são execução única.

Ferramentas em `scripts/`:

- `mock_dry_run.py`: demonstração sem rede.
- `validate.py`: agrega testes e verificações locais em `artifacts/`.
- `review_repository.py`: revisão estática, links, extração textual dos PDFs e diff

sanitizado.

- audit_security.py: auditoria de arquivos/histórico Git local, sem rotação de credenciais nem alteração de histórico.
- render_pdfs.py: gera cópias textuais dos Markdown; não é uma apresentação diagramada.

O legado `imc/legacy/netbox_client.py.disabled` permanece inativo. `imc/teste-home` é um aviso no lugar de um dump antigo. Relatórios históricos em `docs/` não comprovam a validação desta versão. O Markdown atual é a referência; os cinco PDFs da raiz são cópias textuais regeneradas na entrega offline.

Documentação complementar

- Contrato técnico (`SINCRONIZACAO_IMC_NETBOX.md`)
- Instalação (`COMO_INSTALAR_E_TESTAR.md`)
- Mapa de arquivos (`LISTA_DE_ARQUIVOS.md`)
- Requisitos de acesso (`Requisitos-Permissoes-Dependencias-Servidor-Linux-IMC-NetBox.md`)
- Segurança (`docs/SEGURANCA.md`)

A rotação das credenciais anteriormente expostas e a validação real do laboratório ainda dependem do ambiente. Os scripts não registram essas ações como concluídas automaticamente.